



- * System architecture & stack
- * PostgreSQL database schema (ERD)
- * UML: use cases, classes, sequences
- * REST API catalogue
- * Frontend pages by role
- * AI services (matching, tasks, gap analysis)
- * Deployment topology

1. Vision & Problem

Duo connects clients, mentors (freelancers) and learners (apprenants) around real projects. Clients publish missions; mentors deliver work and supervise learners; learners build a verifiable portfolio on authentic tasks. AI assists matching, task breakdown, gap detection and future review flows.

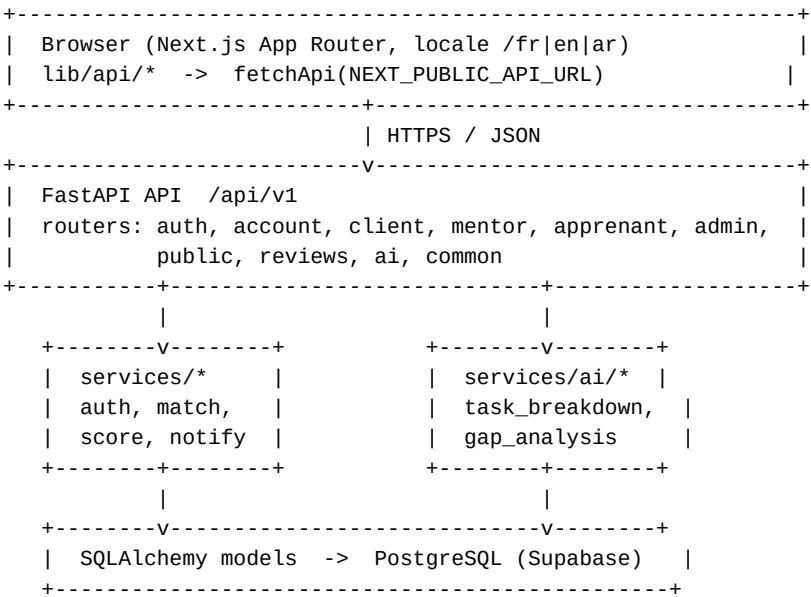
1.1 User roles

- Client -> publishes projects, assigns mentor, validates delivery
- Mentor -> takes projects, recruits learners, delegates tasks, reviews work
- Apprenant -> applies to listings, executes assigned tasks, buys mentor packs
- Admin -> users, categories, disputes, boost/premium configuration

2. Technology Stack

- Frontend Next.js 16 + TypeScript + Tailwind CSS + next-intl (FR/EN/AR, RTL)
- Backend FastAPI (Python 3.11+) + SQLAlchemy + Alembic
- Database PostgreSQL on Supabase (DB only, no Supabase Auth)
- Auth JWT (access + refresh), role-based guards
- AI OpenAI / Groq via AIProvider abstraction
- Deploy Vercel (frontend), Render (backend), Supabase (DB)

3. System Architecture



4. Database Schema (PostgreSQL)

4.1 Core entities

```
users (id PK, email UK, username UK, role, display_name, locale, ...)
|-- client_profiles (user_id FK 1:1)
|-- mentor_profiles (user_id FK 1:1, score, skills JSONB, ...)
|   +-- mentor_categories (M:N with categories)
+-- apprenant_profiles (user_id FK 1:1, skills, career_goal, score)

categories (slug UK, name_fr/en/ar)
skills (slug UK, category_id FK optional)

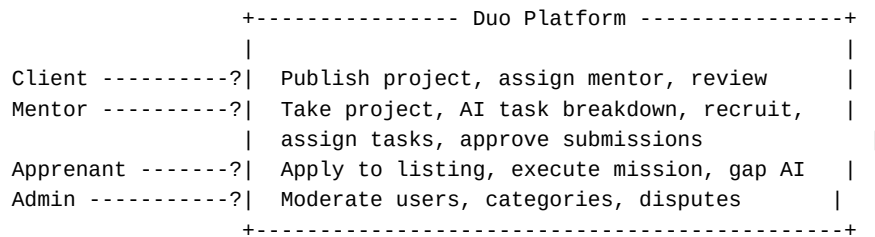
projects (client_id FK, mentor_id FK?, category_id FK, status, budget_dzd,
          required_skills JSONB, learner_complexity_level, description_format)
|-- tasks (project_id FK, assigned_apprenant_id FK?, status, acceptance_criteria)
|   +-- submissions (task_id FK)
|-- deliverables (project_id FK)
+-- listings (mentor_id FK, project_id FK?, required_skills, status)
    +-- applications (listing_id FK, apprenant_id FK, ai_match_score)

reviews (from_user_id, to_user_id, project_id?, type, rating, comment)
mentor_packs -> pack_purchases (apprenant <-> mentor)
boost_options / premium_plans -> subscriptions
notifications, messages, disputes
```

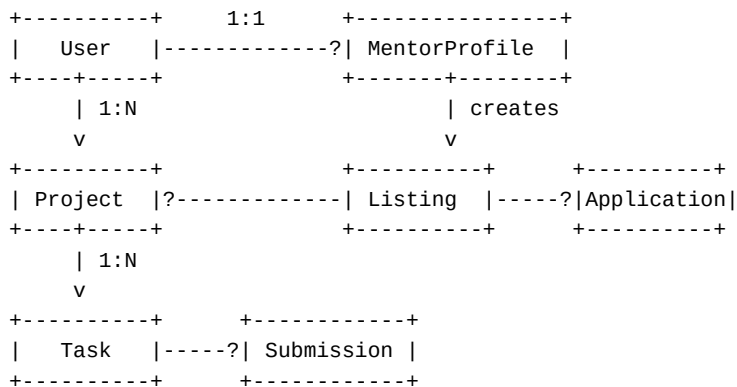
4.2 Enumerations (selected)

```
user_role: client | mentor | apprenant | admin
project_status: draft -> published -> assigned -> in_progress -> delivered -> completed
task_status: todo | in_progress | submitted | revision | approved
listing_status: draft | open | closed | archived
application_status: pending | accepted | rejected | withdrawn
review_type: client_to_mentor | mentor_to_apprenant | apprenant_to_mentor
```

5. UML - Use Cases (summary)



5.1 Class diagram (domain core)



5.2 Sequence - Client assigns mentor (5.1 matching)

```

Client -> GET /client/projects/{id}/mentor-matches
<- match_service: 50% skills overlap + 50% mentor score
Client -> POST /client/projects/{id}/assign-mentor { mentor_id }
-> project.mentor_id set, status -> assigned
-> notification_service -> mentor notified

```

5.3 Sequence - AI gap analysis (5.4)

```

Apprenant -> GET /ai/apprenant/gap-analysis
-> build_learner_gap_context (missions, reviews, catalogue)
-> AIProvider.complete_json_schema (locale-aware prompt)
-> guardrails: filter UUIDs; if catalogue non-empty -> ideas=[]
<- skill_gap, recommended_project_ids, explanation,
suggested_project_ideas (only if catalogue empty)

```

6. REST API Catalogue (/api/v1)

AUTH

POST /auth/register | /login | /refresh GET /auth/me

ACCOUNT

GET/PATCH /account PATCH /{role}-profile POST /onboarding/complete

PUBLIC

GET /categories /mentors /mentors/{id} /apprenants/{id} /listings

CLIENT

GET /dashboard /projects POST /projects PATCH /projects/{id}

POST /projects/{id}/publish

GET /projects/{id}/mentor-matches POST .../assign-mentor

MENTOR

GET /dashboard /projects /projects/disponibles

POST /projects/{id}/take GET/PATCH tasks, listings, applications, packs

POST /ai/projects/{id}/task-breakdown (+ /apply)

APPRENTANT

GET /dashboard /decouvrir/listings /missions /packs /progression

POST /applications GET /ai/apprenant/gap-analysis

ADMIN

GET /dashboard /utilisateurs /litiges /configuration/categories

COMMON

GET/POST /messages GET/PATCH /notifications

7. Frontend Pages (Next.js)

Public: /, /mentors, /mentors/[id], /annonces/[id], /apprenants/[id], /legal

Auth: /auth/connexion, /auth/inscription, /auth/mot-de-passe

Client: /client/dashboard, /client/projets, /client/projets/nouveau,
 /client/projets/[id], /client/onboarding

Mentor: /mentor/dashboard, /mentor/projets, /mentor/projets/[id],
 /mentor/recrutement, /mentor/apprenants, /mentor/packs, /mentor/boost

Apprenant: /apprenant/dashboard, /apprenant/decouvrir, /apprenant/missions,
 /apprenant/progression, /apprenant/packs, /apprenant/profil

Admin: /admin/dashboard, /admin/utilisateurs, /admin/litiges,
 /admin/configuration, /admin/boost-premium

Shared: /account, /messages, /notifications

8. AI Capabilities

- 5.1 Matching match_service - skills + mentor score (client->mentor)
- 5.2 Task break task_breakdown - LLM JSON schema, 3-8 subtasks
- 5.3 Review (planned) submission pre-check vs acceptance criteria
- 5.4 Gap analysis gap_analysis - skill_gap, project recommendations,
 fictional project ideas when catalogue empty

Mentor score formula (5bis.2): Bayesian priors, weights 25/15/35/25
Triggers: review created, project completed, task approved

9. Security & Deployment

JWT in httpOnly-style cookies (duo_access_token, duo_refresh_token)
Role guards on routers; Pydantic validation on all I/O
CORS_ORIGINS whitelist; secrets in env (never committed)

Production topology:

```
[User] -> Vercel CDN (Next.js) -> Render Web Service (FastAPI)
                                     ?
                                     Supabase PostgreSQL
                                     Groq/OpenAI (AI calls)
```

10. Repository & Links

GitHub: <https://github.com/mamanimaniii-a11y/5ibra>

Regenerate this PDF: `python scripts/generate_prototype_pdf.py`

Gap analysis test matrix: `python backend/scripts/test_gap_analysis_matrix.py --prepare`